Tutorial: Drawing a Bounding Box with OpenCV

In this tutorial, you will learn how to use the OpenCV library in Python to load an image and draw a bounding box around an object. This is a fundamental step before moving on to object detection with YOLO.

Step 1: Install OpenCV

If you don't have OpenCV installed, you can install it using pip:

In [ ]:
```
%pip install --upgrade opencv-python
%pip install --upgrade matplotlib
```

Verify the installed version number:

In [ ]:
```
%pip show opencv-python
```

Step 2: Import Required Libraries

Import the libraries you will use:

In [2]:
```
import cv2
from matplotlib import pyplot as plt
```

Step 3: Load the Image

Use OpenCV's `imread` function to load your image.

Replace the path with the location of your image file.

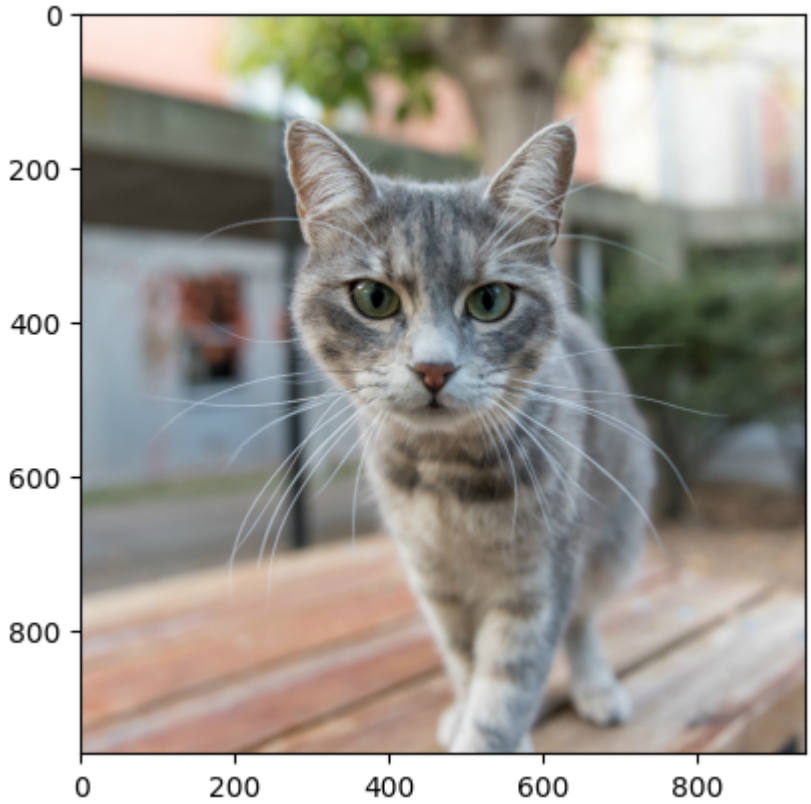
In [7]:
```
# Load the image (change the path to your image file)
image_bgr = cv2.imread('/Users/eugenio/Documents/Computer_Vision/UC.04 Object De
```

In [8]:
```
# Print the shape of the image (height, width, channels)
print('Image shape:', image_bgr.shape)
```

```
Image shape: (960, 940, 3)
```

Step 4: Show the image

In [9]:
```
# Convert BGR to RGB for displaying with matplotlib
image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

# Display the image
plt.imshow(image_rgb)
plt.show()
```



Step 5: Create the Bounding Box over the Image

To draw a bounding box, use OpenCV's `rectangle` function.

You need to specify the coordinates of the **top-left** point $(x1, y1)$ and the **bottom-right** point $(x2, y2)$ of the bounding box.

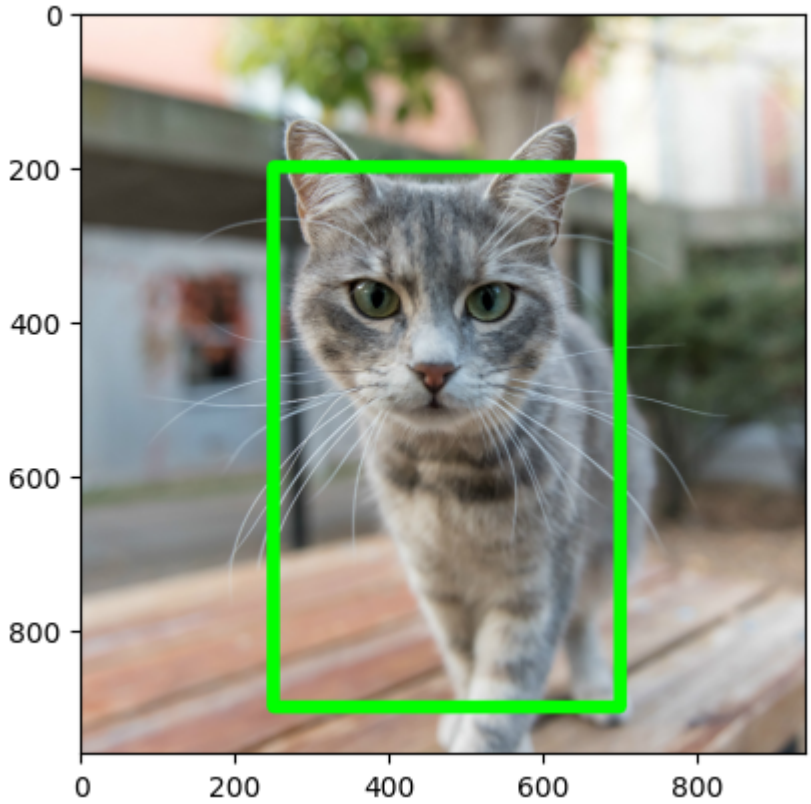
The coordinates should be within the image dimensions

In [14]:
```
x1, y1 = 250, 200  # Top-left corner
x2, y2 = 700, 900  # Bottom-right corner

# Draw the bounding box on the input image using cv2.rectangle
# Arguments: image, (x1, y1), (x2, y2), bounding box color in BGR, thickness
image_bgr_box = cv2.rectangle(image_bgr, (x1, y1), (x2, y2), (0, 255, 0), 15)  #
```

Step 6: Display the Image and Bounding Box

Finally, use matplotlib to display the image with the bounding box.

In [15]:
```
# Convert the image from BGR (OpenCV format) to RGB (matplotlib format)
image_rgb_box = cv2.cvtColor(image_bgr_box, cv2.COLOR_BGR2RGB)

# Display the image
plt.imshow(image_rgb_box)
plt.show()
```



You should now see your image with a bounding box drawn around the object.

You can adjust the bounding box coordinates as needed to fit the object you want to highlight.

Individual Activity:

1. Load an image containing **at least three different objects from different classes** using `cv2.imread`.

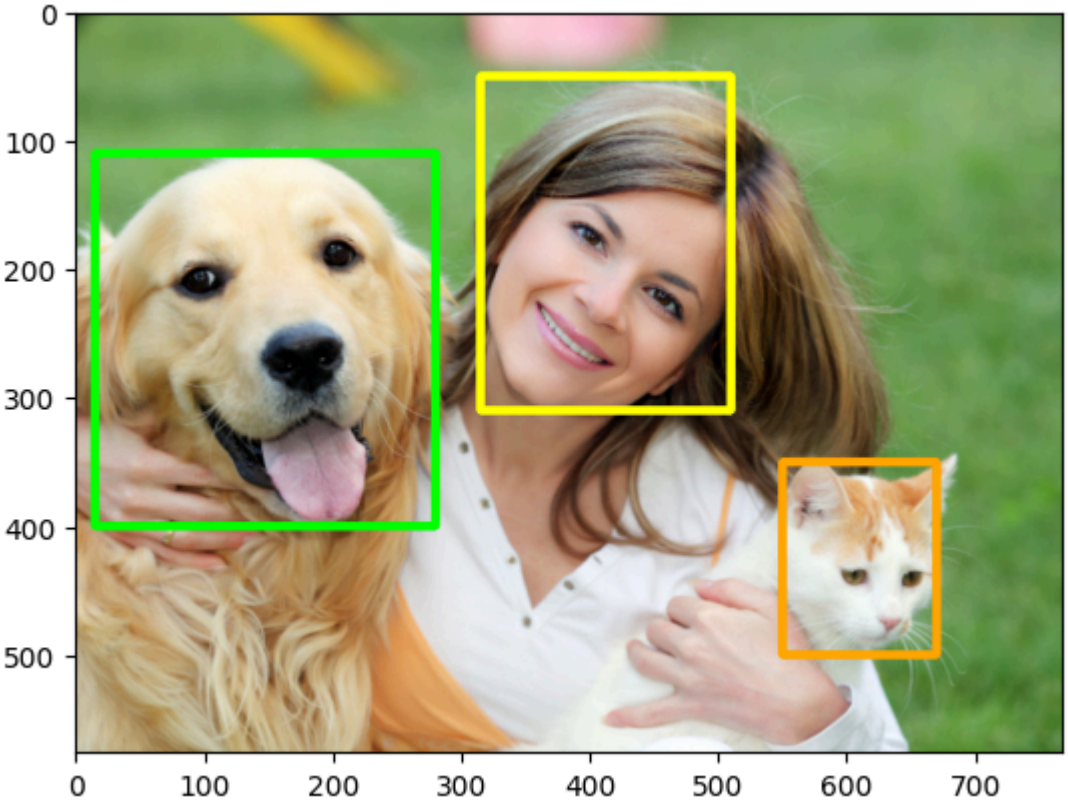
In [4]:
```
# Load the image (change the path to your image file)
image_bgr = cv2.imread(r'semana3\img1.png')
print('Image shape:', image_bgr.shape)
```

```
Image shape: (576, 768, 3)
```

2. Draw **different colored bounding boxes** around each object (at least one of the bounding boxes should be yellow). Use a **line thickness** of 5 for both bounding boxes.

In [5]:
```
### Dibujar caja para perro
x1, y1 = 15, 110 # Top-left corner
x2, y2 = 280, 400 # Bottom-right corner

### Dibujar caja para persona
x3, y3 = 315, 50 # Top-left corner
x4, y4 = 510, 310 # Bottom-right corner

### Dibujar caja para gato
x5, y5 = 550, 350 # Top-left corner
```

```
x6, y6 = 670, 500 # Bottom-right corner
# Draw the bounding box on the input image using cv2.rectangle # Arguments: imag

image_bgr_box_perro = cv2.rectangle(image_bgr, (x1, y1), (x2, y2), (0, 255, 0),
image_bgr_box_persona = cv2.rectangle(image_bgr_box_perro, (x3, y3), (x4, y4), (
image_bgr_box_gato = cv2.rectangle(image_bgr_box_persona, (x5, y5), (x6, y6), (0

# Convert the image from BGR (OpenCV format) to RGB (matplotlib format)
image_rgb_box = cv2.cvtColor(image_bgr_box_gato, cv2.COLOR_BGR2RGB)

# Display the image
plt.imshow(image_rgb_box)
plt.show()
```



3. Add **text labels** above each bounding box (e.g., “Cat”, “Ball”, etc.) using `cv2.putText`.

In [6]:
```
# Dibujar cajas
image_bgr_box_perro = cv2.rectangle(image_bgr, (x1, y1), (x2, y2), (0, 255, 0),
image_bgr_box_persona = cv2.rectangle(image_bgr_box_perro, (x3, y3), (x4, y4), (
image_bgr_box_gato = cv2.rectangle(image_bgr_box_persona, (x5, y5), (x6, y6), (0

# Agregar etiquetas (texto)
font = cv2.FONT_HERSHEY_SIMPLEX

# Perro
cv2.putText(image_bgr_box_gato, 'Perro', (x1, y1 - 10),
            font, 1, (0, 255, 0), 2, cv2.LINE_AA)

# Persona
cv2.putText(image_bgr_box_gato, 'Persona', (x3, y3 - 10),
            font, 1, (0, 255, 255), 2, cv2.LINE_AA)
```

```
# Gato
cv2.putText(image_bgr_box_gato, 'Gato', (x5, y5 - 10),
            font, 1, (0, 165, 255), 2, cv2.LINE_AA)
```

Out[6]:

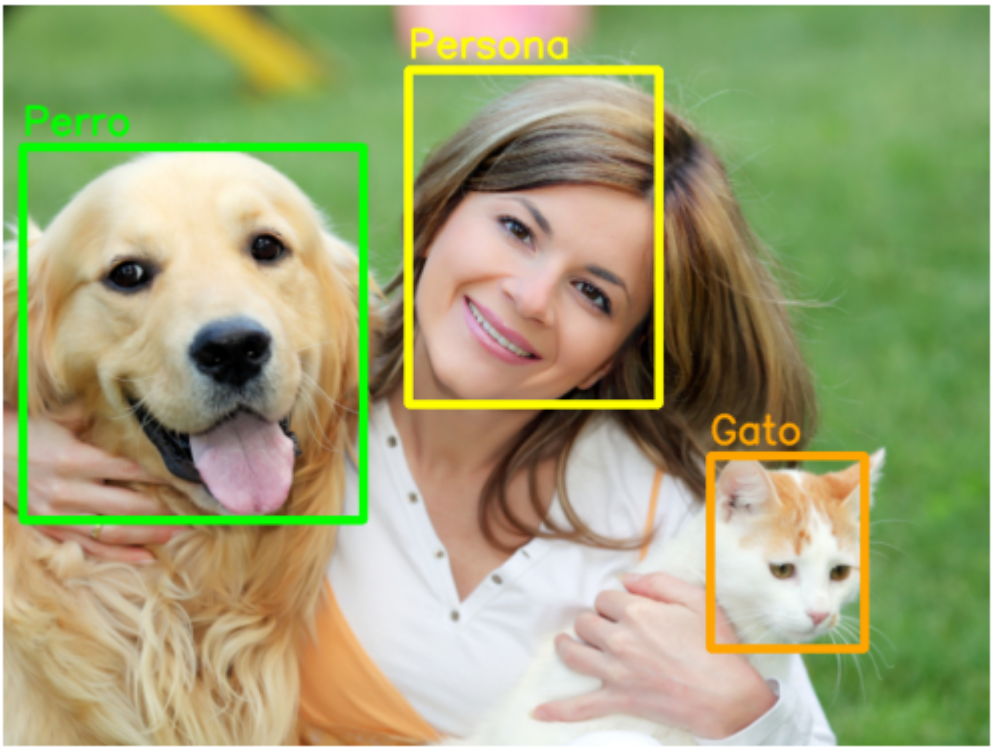
```
array([[[ 23,  50,  42],
        [ 24,  53,  37],
        [ 28,  58,  41],
        ...,
        [ 85, 147,  99],
        [ 84, 146, 101],
        [ 85, 145, 104]],

       [[ 26,  54,  40],
        [ 27,  58,  38],
        [ 30,  63,  43],
        ...,
        [ 85, 146, 103],
        [ 85, 146, 101],
        [ 85, 146, 102]],

       [[ 27,  56,  41],
        [ 31,  62,  42],
        [ 34,  69,  48],
        ...,
        [ 85, 145, 102],
        [ 84, 144, 103],
        [ 85, 146, 103]],

       ...,

       [[188, 205, 208],
        [186, 206, 208],
        [184, 205, 206],
        ...,
        [ 64, 134,  85],
        [ 66, 135,  84],
        [ 69, 135,  86]],

       [[186, 206, 207],
        [184, 205, 206],
        [186, 207, 208],
        ...,
        [ 61, 130,  85],
        [ 63, 133,  85],
        [ 67, 134,  83]],

       [[186, 207, 208],
        [183, 204, 205],
        [184, 205, 206],
        ...,
        [ 57, 129,  85],
        [ 61, 131,  83],
        [ 66, 131,  83]]], dtype=uint8)
```

4. Display the **resulting image**.

In [7]:

```
# Convertir a RGB para mostrar
image_rgb_box = cv2.cvtColor(image_bgr_box_gato, cv2.COLOR_BGR2RGB)
```

```
# Mostrar imagen
plt.imshow(image_rgb_box)
plt.axis('off')
plt.show()
```



5. Suggest one method (written, no code) to **detect objects automatically** in an image (no manual coordinates). Name the model or technique you would use and briefly explain how it produces bounding boxes.

Para la detección automática de objetos o en este caso personas, gatos y perros es posible utilizar modelos ya entrenados con miles de imagenes que utilizan modelos de deepearning o machine learning con buenos resultados. Por otro lado, también es posible entrenar un modelo propio, sin embargo, es posible que no puedan superar a los modelos ya existentes dado que utiliza grandes cantidades de imagenes en entrenamiento, cualquiera de las dos opciones dependerian del objetivo de la detección.

Grading Rubric:

Criteria	Points	Description
1. Image Loading	1	Successfully loads an image with at least three objects using `cv2.imread`. Correct file path and proper handling of image object.
2. Bounding Boxes	3	Draws **different colored bounding boxes** around each object. Boxes have correct size and position, and line thickness is set to 5.
3. Text Labels	2	Adds **accurate text labels** above each bounding box using `cv2.putText`. Labels are readable and positioned correctly.

Criteria	Points	Description
4. Automatic Detection Suggestion	4	Provides a clear and reasonable method to detect objects automatically. Shows understanding of dynamic object detection.

Total: 10 points

Prepared by:
Manuel Eugenio Morocho-Cayamcela